

Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática

Arquitetura de Sistemas Embutidos (ASE)

Sistema de Climatização Automático

ESP32-C6 · ESP-IDF

Autores

Hugo Dias — 114142

Pedro Melo — 114208

Docentes:

Arnaldo Silva Rodrigues de Oliveira
Fábio Daniel Lopes Coutinho

6 de junho de 2026

Resumo

Este relatório descreve o **Sistema de Climatização Automático**, um sistema embebido desenvolvido sobre o microcontrolador **ESP32-C6** com **ESP-IDF**. O sistema monitoriza continuamente a temperatura e humidade ambiente através de um sensor DHT20 (I2C) e atua automaticamente sobre duas ventoinhas, uma para simular ar frio e outra ar quente, de forma a manter a temperatura dentro de uma zona de conforto configurável. Inclui interface local (LCD, botão e potenciômetro), persistência em cartão microSD (configurações e histórico), um servidor web para monitorização e controlo remoto, e tira partido dos modos de baixo consumo do microcontrolador. A arquitetura de software foi organizada em torno de um *controller* central sobre FreeRTOS, separando estado de lógica.

1 Propósito e Objetivos

O propósito do projeto é construir um sistema de regulação térmica autónomo, energeticamente eficiente e observável tanto localmente como pela rede. Os principais objetivos são:

- Monitorizar a temperatura ambiente em tempo real;
- Atuar automaticamente sobre o ambiente com base em limites definidos;
- Minimizar o consumo energético através de modos de baixo consumo;
- Disponibilizar informação ao utilizador através de interface visual;
- Armazenar dados históricos de temperatura;
- Implementar conectividade de rede com um propósito funcional (controlo remoto).

2 Funcionamento e Casos de Uso

O comportamento do sistema segue uma lógica de **histerese** em torno de uma zona de conforto delimitada por T_{min} e T_{max} (valores por omissão 25 °C e 30 °C, ajustáveis):

- Se $T > T_{max}$: liga a ventoinha de arrefecimento até $T \leq T_{max} - 2$;
- Se $T < T_{min}$: liga a ventoinha de aquecimento até $T \geq T_{min} + 2$;
- Se $T_{min} \leq T \leq T_{max}$: zona de conforto, atuadores desligados e sistema em modo de baixo consumo.

A margem de 2 °C evita comutações rápidas dos atuadores junto aos limites. Existem dois atores principais: o **utilizador local**, que consulta o LCD e usa o botão e o potenciômetro para navegar no menu e ajustar limites/definições; e o **utilizador remoto**, que através da página web consulta o estado, liga/desliga o sistema e lê o histórico. A Figura 1 mostra a máquina de estados térmica que materializa este caso de uso central.

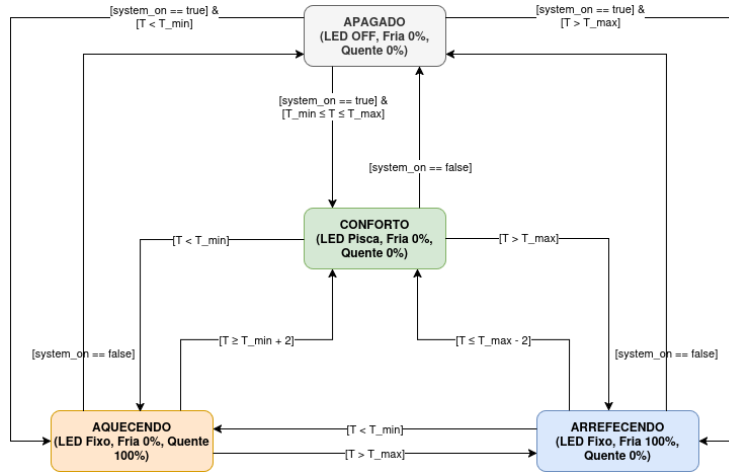


Figura 1: Máquina de estados do controlo térmico. O estado APAGADO corresponde ao sistema desligado; CONFORTO mantém os atuadores parados e o LED a piscar; AQUECENDO/ARREFECENDO acionam a ventoinha respetiva a 100% com o LED fixo.

3 Arquitetura do Sistema

A arquitetura organiza-se à volta de um **controller central**, que funciona como único cérebro do sistema (Figura 2). Cada periférico é encapsulado num módulo. O *input* local (botão e potenciômetro) liga-se diretamente ao módulo de interface; já a comunicação entre os módulos de **lógica e atuação** passa sempre pelo controller (por *comandos*) ou pela leitura do estado partilhado.

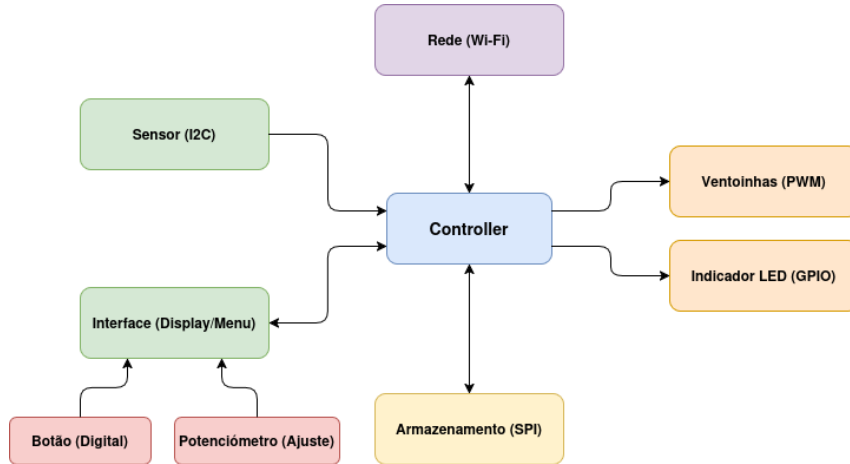


Figura 2: Arquitetura de blocos do sistema. O *Controller* concentra a decisão e liga-se ao sensor (I2C), à interface (botão + potenciômetro + LCD), à rede Wi-Fi, às ventoinhas (PWM), ao indicador LED (GPIO) e ao armazenamento (SPI).

As responsabilidades dividem-se em camadas:

- **Controlador** — cérebro único. Decide o estado térmico e o modo de energia, comanda atuadores (LED e ventoinhas), atualiza o estado partilhado e envia pedidos de escrita ao módulo de armazenamento (via `persist_queue`).
- **Conectividade (Wi-Fi / HTTPS)** — servidor web em modo *Station*. Serve uma página HTML (temperatura, humidade, estado, limites, intervalo de leitura, botão ON/OFF e os últimos registos) e uma pequena API REST para consultar o estado e alterar configurações. Lê o estado por *snapshot* e atua apenas através de comandos; o Wi-Fi só se desliga no menu local.
- **Armazenamento (microSD / SPI)** — driver puro de cartão. Gere o `config.csv` (definições persistentes: T_{max} , T_{min} , deep sleep, Wi-Fi, sistema ON/OFF e intervalo de leitura) e o `logs.csv` (histórico de temperatura e eventos). As escritas ocorrem fora do caminho crítico, consumidas de uma fila própria.
- **Sensor e atuadores** — drivers de um único periférico: o DHT20 (sensor), as ventoinhas (PWM) e o LED (atuadores). O botão e o potenciômetro são lidos pelo módulo de interface, que traduz a interação do utilizador em comandos.

Discussão

Centralizar toda a lógica de decisão (térmica e energética) num único ponto torna o comportamento do sistema previsível e fácil de raciocinar: existe um só lugar onde o estado é escrito. Como o controller é o único escritor do estado partilhado, as restantes *tasks* lêem sempre cópias consistentes (sem condições de corrida), e a redução da comunicação a apenas dois mecanismos, comandos e *snapshot* do estado (Secção 4), mantém os módulos desacoplados e independentes entre si. Esta separação clara entre estado, lógica e periféricos facilita testar e estender cada módulo isoladamente.

4 Arquitetura de Software (FreeRTOS)

O software organiza-se em *tasks* independentes do FreeRTOS, uma por módulo, que comunicam através de **apenas dois mecanismos**: *comandos* (filas) e *estado partilhado* (snapshot sob *mutex*). A Figura 3 ilustra esta organização.

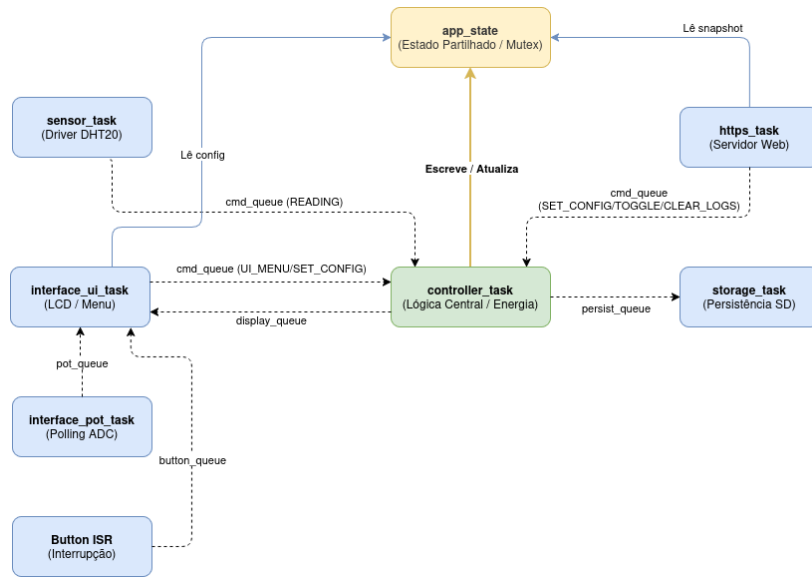


Figura 3: Arquitetura de software: *tasks*, filas (setas tracejadas) e o estado partilhado **app_state**. O **controller_task** é o único escritor de facto do estado; as restantes *tasks* apenas lêem *snapshots*.

Task	Função	Stack	Prioridade
ctrl	Lógica central (térmica + energia)	4096	5
sensor	Driver DHT20 (leitura periódica)	4096	5
sd	Persistência em microSD	6144	4
ui	LCD + menu	5120	4
pot	Leitura do potenciômetro (ADC)	2048	3
https	Servidor web	8192	3

Tabela 1: *Tasks* do sistema, dimensão de *stack* e prioridade.

Comunicação por comandos. O **controller_task** tem uma única fila de entrada de comandos (**controller_cmd_t**), enviados por qualquer produtor através de **controller_post()** de forma não bloqueante. Cada comando tem um efeito bem definido: **READING** grava a nova leitura no estado, regista-a no log e empurra-a para o LCD; **SET_CONFIG** aplica novas definições e, se marcado como persistente, grava o **config.csv**; **TOGGLE_SYSTEM** inverte o estado ligado/desligado; **UI_MENU** sinaliza a entrada/saída do menu, mantendo o sistema em modo *Active* enquanto o utilizador navega; e **CLEAR_LOGS** pede a limpeza do **logs.csv**. O controller publica ainda em filas de saída, **persist_queue** (escritas no SD) e **display_queue** (atualização do LCD), enquanto a interação local chega à *task* de interface pelas filas **button_queue** e **pot_queue**.

Estado partilhado e *mutex*. O **app_state** é um *store* que agrega **config**, estado térmico, modo de energia e a última leitura. Todos os acessos são serializados por um *mutex* (**xSemaphoreTake/Give**): o controller é o único escritor e qualquer *task* pode ler cópias consistentes. Assim, garante-se que nenhuma *task* lê o estado a meio de uma atualização, evitando condições de corrida.

Ciclo do controller. A *task* bloqueia em **xQueueReceive** com um *timeout* de 20s; ao receber um comando trata-o (**handle_cmd**) e, em seguida, executa **reevaluate()**, que recalcula o estado térmico, comanda os atuadores e decide o modo de energia. O *timeout* garante reavaliação periódica mesmo sem comandos (necessário para entrar em *sleep* no estado de repouso).

Gestão de energia. O sistema usa quatro modos: **Active** (CPU e Wi-Fi ligados), **Modem Sleep** (rádio Wi-Fi dorme entre *beacons*, PWM ativo), **Light Sleep** (CPU pausado, RAM mantida) e **Deep Sleep** (apenas RTC, reboot ao acordar). O modo **Active** é reservado a MENU/EDITAR, por exigir ADC responsivo. Fora do conforto (a atuar) ou com Wi-Fi ligado usa-se **Modem Sleep**; em conforto com Wi-Fi desligado, escolhe-se **Light** ou **Deep Sleep** consoante a definição. No Deep Sleep, o *wakeup* é feito por temporizador RTC (igual ao intervalo de leitura) e pelo botão (**esp_deep_sleep_enable_gpio_wakeup**, dado que o ESP32-C6 não suporta **ext0**); a configuração sobrevive por ser relida do **config.csv** a cada arranque.

5 Hardware

A Figura 4 apresenta o esquemático das ligações ao ESP32-C6. O sensor DHT20 liga por I2C (GPIO 6/7) com *pull-ups* de 4.7 k Ω . O Display ST7735 e o cartão microSD partilham o barramento SPI (SCK, MOSI, MISO e linhas de *chip select* distintas), com *pull-ups* de 10 k Ω . As duas ventoinhas são acionadas por PWM (GPIO 4/5, alimentação a 5 V com díodo de proteção), o LED de estado liga ao GPIO 8 (resistência de 180 Ω), e a interação local usa um botão (GPIO 0) e um potenciômetro lido por ADC (GPIO 1).

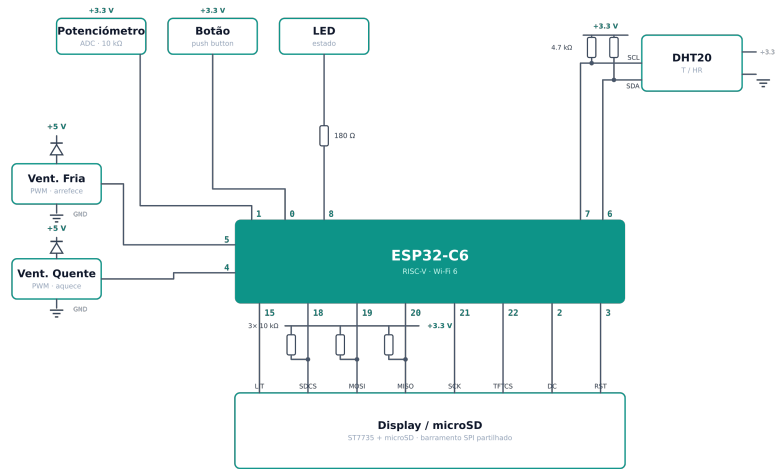


Figura 4: Esquemático das ligações dos periféricos ao ESP32-C6.

6 Resultados e Trabalho Futuro

Atingido: controlo térmico por histerese com atuação automática; interface local completa (LCD, menu navegável por potenciômetro, *timeout* de inatividade); persistência em microSD (configuração e histórico em CSV); servidor web com leitura de estado e controlo remoto ON/OFF; quatro modos de baixo consumo; e uma arquitetura assente num *controller* central com estado *thread-safe*.

Melhorias futuras: controlo PWM variável nas ventoinhas (atualmente a 100% fixo); interface web com atualizações em tempo real via WebSockets/SSE; suporte a múltiplos sensores ou zonas independentes; integração MQTT para compatibilidade com plataformas de *home automation* (e.g. Home Assistant).

7 Reutilização de Código e Uso de LLMs

O desenvolvimento partiu de exemplos oficiais do ESP-IDF, adaptados e integrados como componentes (Tabela 2). Sobre estas bases construiu-se a lógica própria do projeto: a arquitetura *controller/app_state*, as máquinas de estados (térmica, energia e UI), a gestão de modos de baixo consumo e a persistência em CSV.

Módulo	Exemplo ESP-IDF de base	Aula
DHT20	—	Aula 4
LED	<code>get-started/blink</code>	Aula 3
Botão	<code>peripherals/gpio/generic_gpio</code>	Aula 3
Potenciômetro	<code>peripherals/adc/oneshot_read</code>	Aula 3
microSD	<code>storage/sd_card/sdspi</code>	Aula 6
Display TFT	exemplo de TFT (ST7735)	Aula 10
Wi-Fi / HTTPS	<code>protocols/https_server/simple</code>	Aula 8

Tabela 2: Exemplos ESP-IDF reutilizados como ponto de partida de cada módulo.

Uso de LLMs. Claude foi utilizado como ferramenta de apoio ao desenvolvimento: consulta e interpretação de APIs do ESP-IDF, criação da página web de monitorização, reorganização do código para melhorar a sua estrutura e legibilidade, diagnóstico de erros reportados pelo monitor serial, e revisão e compactação deste relatório. O Gemini foi utilizado para converter o esquema de ligações de desenho para imagem. Toda a arquitetura, lógica de controlo e decisões de design foram definidas e implementadas pelos autores.

Referências

- [1] Espressif Systems, *ESP-IDF Programming Guide — Peripherals, Sleep Modes and Wi-Fi*. <https://docs.espressif.com/projects/esp-idf/>
- [2] Aosong, *DHT20 Temperature and Humidity Sensor Datasheet*.
- [3] Sitronix, *ST7735 TFT LCD Controller Datasheet*.